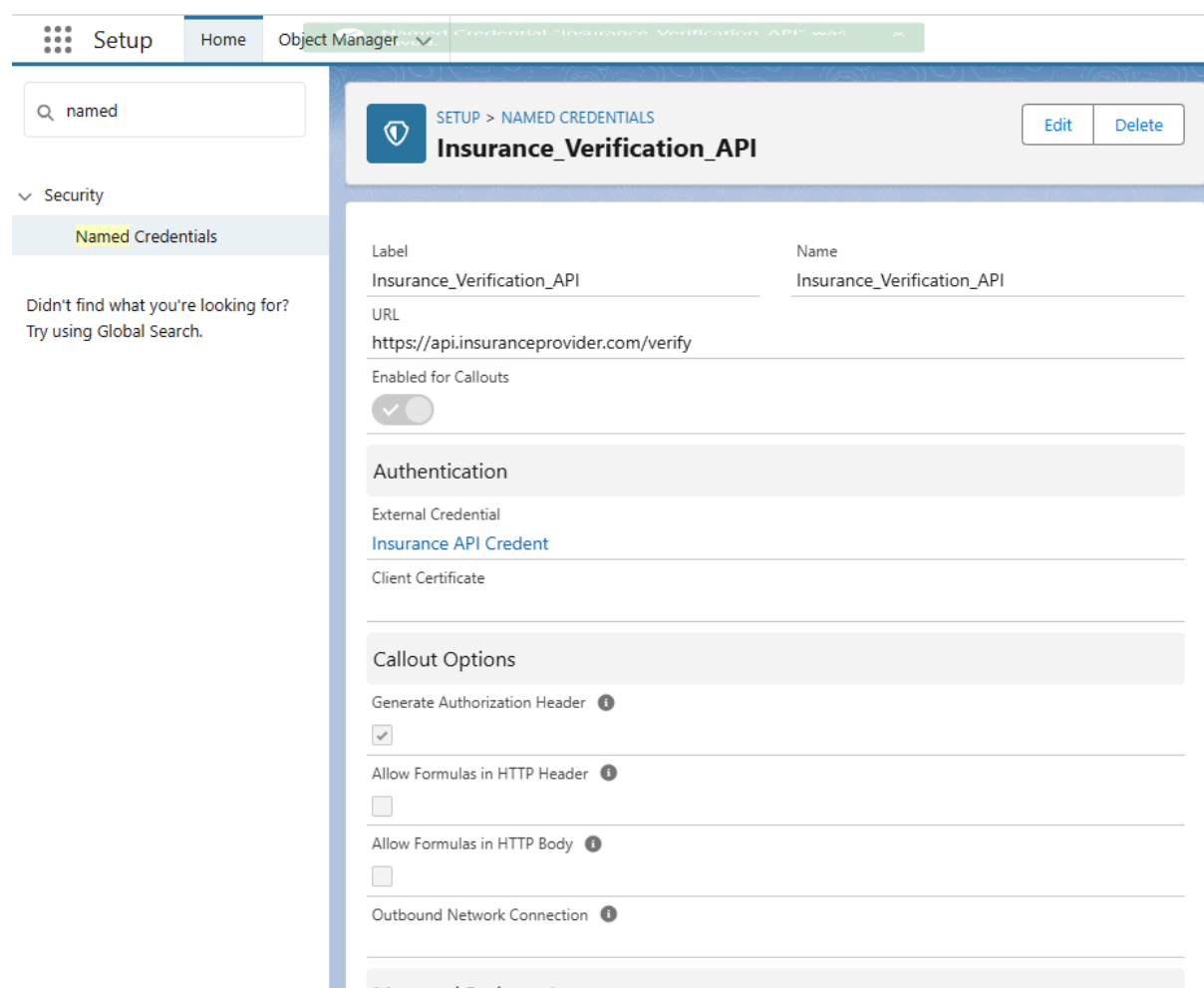


Phase 7: Integration & External Access

Goal: Extend Salesforce to securely connect with external systems such as insurance verification services, warranty portals, and external car management databases.

1. Named Credentials

We configured **Named Credentials** to securely store external API endpoints and authentication details for the Insurance Verification API. This allows Apex callouts to use stored credentials without hard-coding sensitive information, improving both security and maintainability.



The screenshot shows the Salesforce Setup interface. The left sidebar has a search bar with 'named' entered. Under the 'Security' section, 'Named Credentials' is selected. The main content area shows the configuration for a Named Credential named 'Insurance_Verification_API'. The configuration includes a label 'Insurance_Verification_API', a name 'Insurance_Verification_API', a URL 'https://api.insuranceprovider.com/verify', and a toggle for 'Enabled for Callouts' which is turned on. Below this, the 'Authentication' section shows an 'External Credential' link to 'Insurance API Credent' and a 'Client Certificate' field. The 'Callout Options' section includes checkboxes for 'Generate Authorization Header' (checked), 'Allow Formulas in HTTP Header' (unchecked), 'Allow Formulas in HTTP Body' (unchecked), and 'Outbound Network Connection' (unchecked). At the top of the main content area, there is a green banner that reads 'Named Credential "Insurance_Verification_API" was successfully created.'

Label	Name
Insurance_Verification_API	Insurance_Verification_API

URL
https://api.insuranceprovider.com/verify

Enabled for Callouts
☒

Authentication

External Credential
[Insurance API Credent](#)

Client Certificate

Callout Options

Generate Authorization Header 
☒

Allow Formulas in HTTP Header 
☐

Allow Formulas in HTTP Body 
☐

Outbound Network Connection 
☐

2. External Services

We evaluated **External Services** to connect Salesforce declaratively to REST APIs that provide an OpenAPI specification. This would allow automated insurance verification directly from Salesforce Flows when new service appointments are created.

3. Web Services (REST/SOAP)

To enable external partners to check warranty information, we exposed a **custom Apex REST service**. This service provides authorized systems with warranty details (status, product, expiry date) in real time, supporting smooth collaboration with third-party warranty providers.

```
@RestResource(urlMapping='/warranty/*')
```

```
global with sharing class WarrantyRestService {
```

```
    @HttpGet
```

```
    global static Warranty__c getWarranty() {
```

```
        String wld = RestContext.request.requestURI.substringAfterLast('/');
```

```
        try {
```

```
            return [SELECT Id, Product__c, Status__c, Expiry_Date__c
```

```
                    FROM Warranty__c
```

```
                    WHERE Id = :wld];
```

```
        } catch (Exception e) {
```

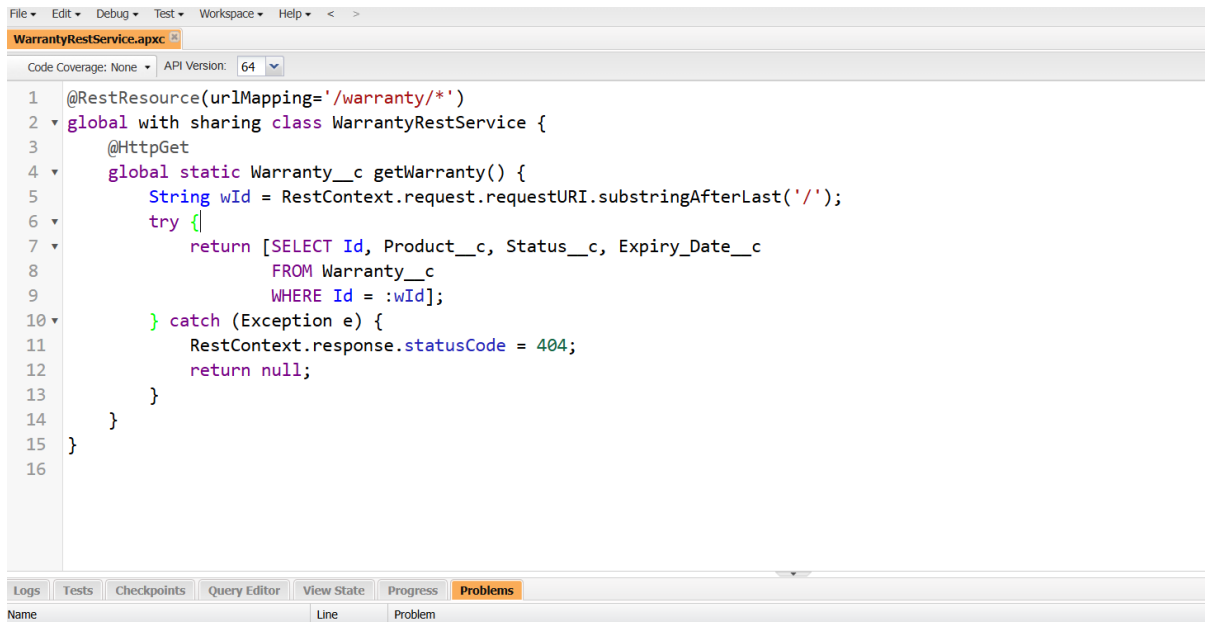
```
            RestContext.response.statusCode = 404;
```

```
            return null;
```

```
        }
```

```
    }
```

```
}
```



```
1 @RestResource(urlMapping='/warranty/*')
2 global with sharing class WarrantyRestService {
3     @HttpGet
4     global static Warranty__c getWarranty() {
5         String wId = RestContext.request.requestURI.substringAfterLast('/');
6         try {
7             return [SELECT Id, Product__c, Status__c, Expiry_Date__c
8                     FROM Warranty__c
9                     WHERE Id = :wId];
10        } catch (Exception e) {
11            RestContext.response.statusCode = 404;
12            return null;
13        }
14    }
15 }
16
```

4. Callouts

We implemented **Apex callouts** to the Insurance Verification API. When a new service appointment is created, Salesforce automatically calls the external insurance system to validate the customer's policy, ensuring appointments are only confirmed for eligible customers.

```
public class InsuranceService {

    @future(callout=true)

    public static void verifyPolicy(String bookingId, String policyNumber) {

        HttpRequest req = new HttpRequest();

        req.setEndpoint('callout:Insurance_Verification_API/policies/' + policyNumber);

        req.setMethod('GET');

        Http http = new Http();

        HttpResponse res = http.send(req);

        if (res.getStatusCode() == 200) {

            // Example: parse JSON and update Service Appointment
```

```

        Service_Appointment__c sa = [SELECT Id FROM Service_Appointment__c WHERE Id =
:bookingId LIMIT 1];

        sa.Status__c = 'Verified';

        update sa;

    } else {

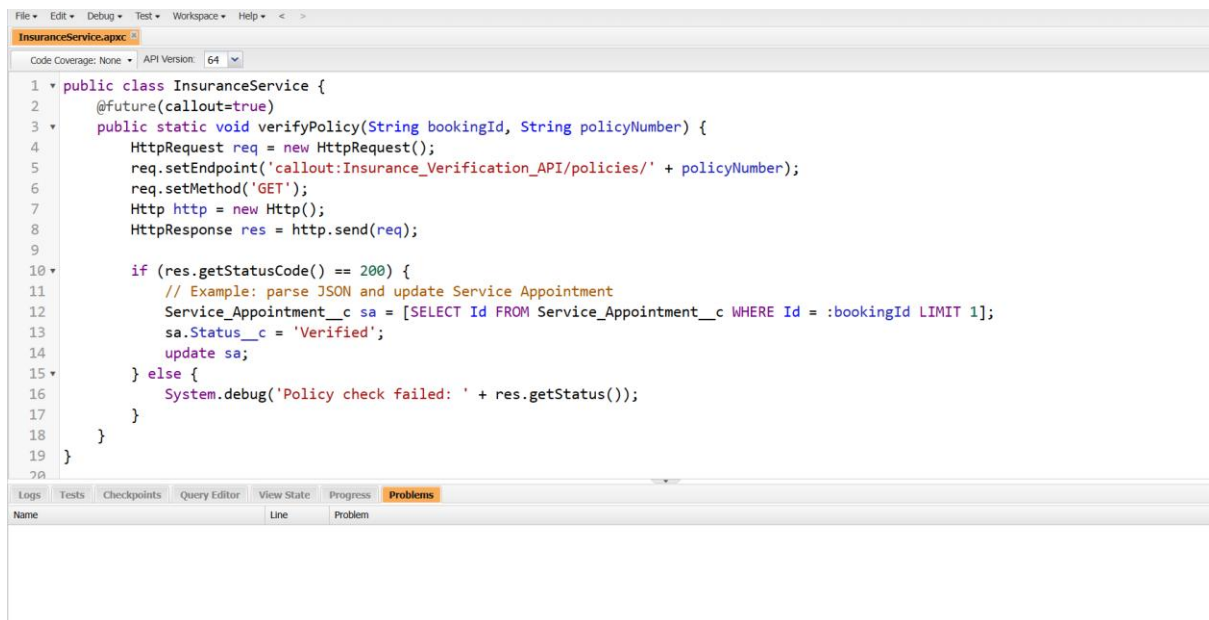
        System.debug('Policy check failed: ' + res.getStatus());

    }

}

}

```



```

1  public class InsuranceService {
2      @future(callout=true)
3      public static void verifyPolicy(String bookingId, String policyNumber) {
4          HttpRequest req = new HttpRequest();
5          req.setEndpoint('callout:Insurance_Verification_API/policies/' + policyNumber);
6          req.setMethod('GET');
7          Http http = new Http();
8          HttpResponse res = http.send(req);
9
10         if (res.getStatusCode() == 200) {
11             // Example: parse JSON and update Service Appointment
12             Service_Appointment__c sa = [SELECT Id FROM Service_Appointment__c WHERE Id = :bookingId LIMIT 1];
13             sa.Status__c = 'Verified';
14             update sa;
15         } else {
16             System.debug('Policy check failed: ' + res.getStatus());
17         }
18     }
19 }

```

5. Platform Events

We designed a **Platform Event** called Breakdown_Reported__e to notify external roadside assistance partners when a breakdown is logged. This provides real-time, event-driven communication and speeds up response time for critical cases.

```

public class PublishBreakdownEventQueueable implements Queueable {

    private List<Id> appointmentIds;

    public PublishBreakdownEventQueueable(List<Id> appointmentIds) {

        this.appointmentIds = appointmentIds;
    }
}

```

```
}
```

```
public void execute(QueueableContext context) {
```

```
    List<Breakdown_Reported__e> events = new List<Breakdown_Reported__e>();
```

```
    for (Id appld : appointmentIds) {
```

```
        events.add(new Breakdown_Reported__e(
```

```
            Notes__c = 'Breakdown reported for Appointment ' + appld
```

```
        ));
```

```
    }
```

```
    if (!events.isEmpty()) {
```

```
        Database.SaveResult[] results = EventBus.publish(events);
```

```
        for (Database.SaveResult sr : results) {
```

```
            if (!sr.isSuccess()) {
```

```
                System.debug('Event publish failed: ' + sr.getErrors()[0].getMessage());
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
}
```

```

1 public class PublishBreakdownEventQueueable implements Queueable {
2     private List<Id> appointmentIds;
3
4     public PublishBreakdownEventQueueable(List<Id> appointmentIds) {
5         this.appointmentIds = appointmentIds;
6     }
7
8     public void execute(QueueableContext context) {
9         List<Breakdown_Reported__e> events = new List<Breakdown_Reported__e>();
10
11         for (Id appId : appointmentIds) {
12             events.add(new Breakdown_Reported__e(
13                 Notes__c = 'Breakdown reported for Appointment ' + appId
14             ));
15         }
16
17         if (!events.isEmpty()) {
18             Database.SaveResult[] results = EventBus.publish(events);
19             for (Database.SaveResult sr : results) {
20                 if (!sr.isSuccess()) {

```

trigger ServiceAppointmentTrigger_New on Service_Appointment__c (after insert) {

if (Trigger.isAfter && Trigger.isInsert) {

List<Id> breakdownAppointments = new List<Id>();

for (Service_Appointment__c sa : Trigger.new) {

if (sa.Status__c == 'Breakdown') {

breakdownAppointments.add(sa.Id);

}

}

if (!breakdownAppointments.isEmpty()) {

// Enqueue the asynchronous job

System.enqueueJob(new

PublishBreakdownEventQueueable(breakdownAppointments));

}

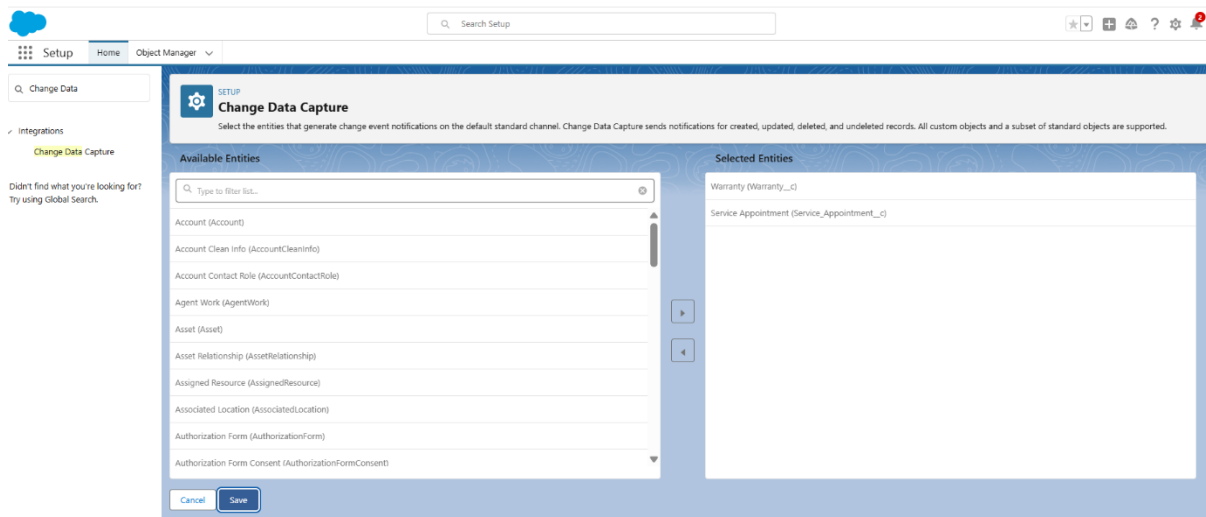
}

}

```
1 trigger ServiceAppointmentTrigger_New on Service_Appointment__c (after insert) {
2     if (Trigger.isAfter && Trigger.isInsert) {
3         List<Id> breakdownAppointments = new List<Id>();
4         for (Service_Appointment__c sa : Trigger.new) {
5             if (sa.Status__c == 'Breakdown') {
6                 breakdownAppointments.add(sa.Id);
7             }
8         }
9         if (!breakdownAppointments.isEmpty()) {
10             // Enqueue the asynchronous job
11             System.enqueueJob(new PublishBreakdownEventQueueable(breakdownAppointments));
12         }
13     }
14 }
15
```

6. Change Data Capture (CDC)

Change Data Capture was enabled for Service Appointments and Warranty objects. This ensures that whenever records are created or updated, external systems receive real-time updates, avoiding the need for constant polling and reducing API consumption.



7. Salesforce Connect

We used **Salesforce Connect** to integrate with an external car inventory database via OData. This allows Salesforce users to view and reference car records in real time without physically importing the data, ensuring accuracy and reducing duplication.

External Data Source: car Appointment

Connect to another Salesforce org or a third-party database or content system.

« Back to External Data Sources

Buttons: Edit, Validate and Sync, Delete

Field	Value
External Data Source	car Appointment
Name	car_Appointment
Type	Salesforce Connect: OData 4.0
URL	https://externalcd.com/car
Connection Timeout (Seconds)	120
Writable External Objects	☐
High Data Volume	☐
Server Driven Pagination	☐
Request Row Counts	☒
Compress Requests	☐
Enable Search	☒
Use Free-Text Search Expressions	☐
Format	JSON
Special Compatibility	None
Display Server Errors	☐
Eligible for External Change Data Capture	☐

8. API Limits

We designed the integration strategy with Salesforce **API limits** in mind. By using Platform Events and Change Data Capture instead of frequent queries, we minimized unnecessary API calls and ensured the system scales efficiently without exceeding limits.

Company Information

Smart Service Appointment & Warranty Automation

The organization's profile is below:

Links: User Licenses (12d) | Permission Set Licenses (12d) | Feature Licenses (11d) | Unsubscribed Entitlements (15d)

Buttons: Edit

Field	Value
Organization Name	Smart Service Appointment & Warranty Automation
Primary Contact	OrgTeam EPIC
Address	India
Fiscal Year Starts In	January
Activate Multiple Currencies	☐
Enable Data Translation	☐
Admin Newsletter	☒
Hide Notices About System Maintenance	☐
Hide Notices About System Downtime	☐
Locale Formats	ICU
Phone	
Default Locale	English (United States)
Default Language	English
Default Time Zone	(GMT+05:30) India Standard Time (Asia/Colombo)
Currency Locale	English (India) - INR
User Data Space	362 MB (7%) Go up
Used File Space	21 MB (0%) Go up
API Requests, Last 24 Hours	860 (15,000 max)
Streaming API Events, Last 24 Hours	0 (10,000 max)
Restricted Logins, Current Month	0 (0 max)
Salesforce.com Organization ID	00DgK000000V14
Organization Edition	Developer Edition
Instance	CAN96
Created By	OrgTeam EPIC, 9/12/2025, 10:28 PM
Modified By	Rhishita Bole, 9/17/2025, 2:11 AM

Buttons: Edit

Name	Status	Total Licenses	Used Licenses	Remaining Licenses	Expiration Date
Salesforce	Active	4	3	1	
Analytics Cloud Integration User	Active	2	2	0	

9. OAuth & Authentication

To secure integrations, we configured a **Connected App** with OAuth 2.0. External warranty and insurance systems authenticate using secure tokens when accessing Salesforce APIs, protecting sensitive customer and appointment data.

The screenshot displays the AWS IAM console interface. At the top, there's a navigation bar with 'Setup', 'Home', and 'Object Manager' tabs. A search bar is located on the right. The left sidebar shows a search bar and a navigation menu with 'Apps' expanded, listing 'App Manager', 'External Client Apps', and 'External Client App Manager'. The main content area is titled 'Manage External Client Apps' and 'Warranty_Portal_Integration'. It shows details for the app, including 'Contact Email' (bhavithabolla2005@gmail.com), 'App Authorization' (All users can self-authorize), 'Type' (Local), and 'App Status' (Enabled). Below this, there are tabs for 'Policies', 'Settings' (selected), and 'Package Defaults'. The 'Settings' tab contains a section for 'Basic Information' with fields for 'External Client App Name', 'Contact Email', 'Contact Phone', 'Icon URL', 'API Name', 'Distribution State' (set to 'Local'), 'Info URL', 'Logo Image URL', and 'Description'.

10. Remote Site Settings

As a legacy fallback, we configured **Remote Site Settings** to whitelist external API domains. However, our primary approach used Named Credentials, which combine endpoint and authentication securely in one place.